

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

AlgoBlocks

(Proyecto DGS)

*Especificación Técnica de Estructuración de Monorepo, Flujo Lógico y
Diseño de Arquitectura*

Para uso de:	Cuerpo de Profesores y Colaboradores
Fecha de Creación:	Junio, 2026
Estado:	Aprobado - Configuración de Monorepo Inicializada

1. Introducción y Contexto del Proyecto

El proyecto AlgoBlocks (DGS) surge como una modernización educativa para la facultad. Anteriormente, los alumnos utilizaban una aplicación de escritorio para comprender los fundamentos de la lógica de programación. Dicho entorno se basaba en la construcción de algoritmos mediante diagramas de bloques (conceptualmente similares a los diagramas de flujo tradicionales, pero con una representación visual más amplia y acotada al flujo de ejecución).

La nueva propuesta migra esta lógica a una aplicación web moderna (SPA), permitiendo a los estudiantes acceder de manera ágil desde cualquier navegador sin necesidad de instalaciones locales. Para esta etapa de arranque, no se implementará base de datos ni registro de usuarios, por lo que toda la persistencia y ejecución de algoritmos se resolverá de forma local en el navegador del alumno. Esto reduce los costos de despliegue a cero y acelera el desarrollo inicial del prototipo funcional.

Los 6 Bloques Lógicos Fundamentales (Modelo Conceptual)

El estudiante compone sus soluciones combinando 6 elementos lógicos que representan las estructuras de datos y control básicas presentes en cualquier lenguaje de programación:

- **1. Input (Entrada):** Permite solicitar datos al usuario para guardarlos en una variable. Ej: `|input: x|`.
- **2. Output (Salida):** Muestra en pantalla un texto literal o el valor de una variable. Ej: `|output: x|` o `|output: "hola"|`.
- **3. Asignación:** Inicializa o actualiza el valor de una variable. Ej: `|asigna: x = 5|` o `|asigna: c = c + 1|`.
- **4. Alternativa (Condicional):** Define una condición lógica. Se divide visualmente en el camino verdadero y camino falso, ejecutando el bloque que corresponda.
- **5. Ciclo Mientras (While):** Repite los bloques en su interior de forma continua mientras se cumpla la condición lógica.
- **6. Ciclo Para (For):** Bucle iterador que ejecuta los bloques en su interior desde un límite inferior (a) hasta un límite superior (b).

2. Estructura de Proyecto Limpia (algotblocks/)

Para mantener el proyecto ordenado y listos para escalar a servicios de almacenamiento compartido más adelante, utilizamos un monorepo básico. Hemos limpiado todas las dependencias pesadas de bases de datos relacionales, ORMs (Prisma) y orquestadores como Docker por el momento, permitiendo que el proyecto sea sumamente liviano y fácil de inicializar.

La estructura actual de directorios en el repositorio es la siguiente:

```
algotblocks/
├── frontend/           # SPA interactiva en React + TypeScript + Vite
│   ├── src/           # Código fuente de componentes, intérprete y hooks
│   ├── public/        # Recursos estáticos del navegador
│   └── package.json    # Dependencias del cliente
├── backend/           # API REST de apoyo en Node.js + Express + TypeScript
│   ├── src/           # Controladores y rutas estructuradas (ej: endpoints futuros)
│   └── package.json    # Dependencias del servidor Express limpio
├── docs/              # Carpeta central de documentación del proyecto
│   ├── arquitectura/   # Decisiones de diseño y este documento PDF
│   ├── casos_de_uso/   # Requerimientos y flujos de usuario
│   └── reuniones/      # Minutas de seguimiento
├── docker/            # Configuraciones futuras de Docker
└── .gitignore         # Exclusiones de Git a nivel monorepo
```

3. Patrones y Estructuras en Frontend y Backend

Aunque no haya una base de datos activa ni autenticación de usuarios, la base del monorepo está construida de manera modular para que sirva de punto de partida sólido y limpio.

Backend: Base Limpia de Express

El backend mantiene el patrón de arquitectura por capas (Controladores, Rutas y Middlewares) pero libre de persistencia por el momento. Está listo para implementar endpoints de compartir código (ej. guardar programas con un ID autogenerated en un JSON local o base de datos ligera) en cuanto se requiera.

Frontend: Estado y Motor de Ejecución local

El cliente React encapsula toda la funcionalidad crítica sin depender del backend. Los puntos clave son:

- **Variables en LocalStorage:** Los bloques y proyectos que el alumno modifique se serializan a JSON y se guardan localmente en el navegador, previniendo pérdidas de datos al refrescar la página.
- **Exportación/Importación:** El sistema admite descargar los programas en archivos planos `.json` para que los estudiantes se los envíen a los profesores o los compartan manualmente.
- **Tipado Estricto de Bloques:** Los tipos en TypeScript mapean la estructura de datos recursiva para facilitar la renderización dinámica en el Canvas de la aplicación y la validación en tiempo de ejecución.
- **Interpreter Engine:** Ubicado en `frontend/src/engine/interpreter.ts`, el motor de ejecución evalúa paso a paso el programa y proporciona asincronicidad, pausándose cuando necesita la respuesta del alumno a un bloque input.